

Java Programming Study Material

Study goal: Learn Java step by step by reading the explanation, typing the examples, and practising each topic.

How to Use These Notes

Read one section at a time. Type each example instead of only reading it, compile it, run it, and change one small part to see what happens.

1. Read the explanation and underline new words.
2. Type every example into a Java file.
3. Compile often so that errors stay small and easy to fix.
4. Use the practice tasks after studying a topic.
5. Use the syntax cheat sheet for quick revision.

Remember: Programming becomes easier through regular practice. Twenty focused minutes each day is better than one long study session once a month.

Contents

Part	Topics
1–5	Java basics, tools, first program, classes, and execution
6–10	Tokens, variables, types, input/output, and operators
11–15	Decisions, loops, methods, arrays, and strings
16–20	Objects, inheritance, packages, exceptions, and collections
21–25	Generics, files, dates, enums, records, and lambdas
26–30	Streams, threads, JDBC, arguments, errors, and debugging
31–35	Best practices, projects, viva questions, practice, and glossary

1. What Is Java?

Java is a general-purpose programming language. It is used to build desktop programs, web services, Android applications, business systems, and many other kinds of software.

History

Java was created at Sun Microsystems in the 1990s. James Gosling and his team designed it to be portable, which means that the same compiled program can run on many operating systems when a suitable Java runtime is available. Java is now maintained by Oracle and the wider Java community.

Important Features

- **Simple:** Java has clear rules and removes many difficult memory tasks.
- **Object-oriented:** Programs can be organised using classes and objects.
- **Portable:** Java code can run on different systems through the Java Virtual Machine.
- **Secure:** Java checks code carefully and does not normally allow direct memory access.
- **Robust:** Strong type checking and exception handling help prevent many errors.
- **Multithreaded:** A program can perform more than one activity at a time.

Advantages, Editions, and Uses

Edition	Simple meaning
Java SE	Core Java features for learning and general programs.
Jakarta EE	Enterprise and web application technologies.
Java ME	Java for small or specialised devices.

Common use	Why Java is useful
Business software	Reliable libraries and clear organisation.
Web services	Strong tools for servers and APIs.
Android development	Large ecosystem and familiar object-oriented syntax.
Education	Good language for learning programming ideas.

2. JDK, JRE, JVM, and Installing Java

The **JDK**, or Java Development Kit, contains tools for writing and compiling Java programs. The **JRE**, or Java Runtime Environment, contains the parts needed to run Java programs. The **JVM**, or Java Virtual Machine, runs Java bytecode on the computer.

Part	Job
JDK	Develop, compile, test, and run Java programs.
JRE	Provide the runtime libraries and JVM.
JVM	Execute Java bytecode.

Basic Setup

1. Install a current Java Development Kit from a trusted Java provider.
2. Open a terminal and check the installation with `java --version`.

3. Check the compiler with `javac --version`.
4. Set the `JAVA_HOME` variable to the JDK folder if your tools need it.
5. Add the JDK `bin` folder to `PATH` so commands work from any folder.

PATH tip: After changing `PATH`, open a new terminal. If the command is still not found, check that the folder is correct and that the JDK, not only a runtime, was installed.

3. First Java Program

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

What Each Line Does

- **public class HelloWorld** creates a class. The file should be named `HelloWorld.java`.
- **public static void main(String[] args)** is the usual starting method of a Java application.
- **System.out.println** prints a message and moves to the next line.
- **Braces** show where the class and method begin and end.

Compiling and Running

```
javac HelloWorld.java  
java HelloWorld
```

The first command creates bytecode in a class file. The second command asks the JVM to run the class. Do not write `.class` after the class name when running it.

4. Class Structure, Comments, and Execution

A Java program can contain packages, imports, a class, fields, constructors, methods, and statements. The compiler checks the source file and produces bytecode. The JVM loads the bytecode, checks it, and executes it.

```
// A line comment  
/* A block comment */  
/** A documentation comment */
```

Comments explain code to people. The compiler does not use normal comments as program instructions. Use comments for useful reasons, not for obvious lines.

5. Java Tokens

A token is a small meaningful part of a Java program.

Token	Meaning	Example
Keyword	A reserved word.	class, if, return
Identifier	A programmer-created name.	total, Student
Literal	A value written directly.	42, "Hi", true
Operator	A symbol that performs an action.	+, ==
Separator	A symbol that organises code.	;, {}, ()
Comment	Notes for people reading code.	// note

How to Use These Notes

Use these notes in small steps. Read one section, type the example yourself, and run it before moving to the next section.

1. Read the explanation slowly and underline new words.
2. Type every code example instead of only reading it.
3. Change one value at a time and observe the result.
4. Solve the practice tasks without looking at the answer first.
5. Use the syntax cheat sheet for quick revision.

Study tip: Programming becomes easier with regular practice. Even 20 minutes each day can build strong skills.

Contents

Part	Topics
1–5	Introduction, compiler, first program, and program basics
6–10	Tokens, variables, data types, input/output, and operators
11–15	Decisions, loops, functions, arrays, and strings
16–20	Pointers, user-defined types, memory, files, and the preprocessor
21–27	Arguments, errors, best practices, projects, viva, practice, and glossary

1. What Is C?

C is a general-purpose programming language. It is used to give clear instructions to a computer. C is small, fast, and close to the way a computer manages memory.

History

C was created by Dennis Ritchie at Bell Laboratories in the early 1970s. It grew from earlier languages such as BCPL and B. The language became popular because it was powerful,

portable, and suitable for system software.

Important Features

- **Simple core:** C has a small set of important keywords and rules.
- **Fast:** C programs usually run quickly because they are compiled into machine code.
- **Portable:** The same source code can often run on different systems after small changes.
- **Structured:** Programs can be divided into functions and clear blocks.
- **Memory control:** Pointers allow direct work with memory addresses.

Advantages and Common Uses

Advantage or use	Simple meaning
Operating systems	Parts of many operating systems are written in C.
Embedded systems	C controls small devices, sensors, and machines.
Compilers	Many language tools are written using C.
Games and graphics	C can provide speed for graphics and game engines.
Learning programming	C teaches memory, logic, and structured problem solving.

2. Installing or Using a Compiler

A compiler changes C source code into a program that the computer can run. Common choices include GCC, Clang, and compiler tools inside an IDE such as Code::Blocks, Dev-C++, or Visual Studio Code with a C extension.

Basic Setup

1. Install a C compiler or open an online C compiler.
2. Create a file with the extension `.c`, such as `main.c`.
3. Write the program and save the file.
4. Compile the source file.
5. Fix errors shown by the compiler.
6. Run the finished program and check its output.

Remember: A compiler error is a message about the source code. Read the line number and the words near the error before changing anything.

3. First C Program

```
#include <stdio.h>
int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

What Each Line Does

- **#include <stdio.h>** adds the standard input/output tools, including `printf`.
- **int main(void)** is the starting function of the program. It returns an integer and takes no input here.
- **printf** displays text. `\n` moves the cursor to a new line.
- **return 0;** tells the operating system that the program ended successfully.
- **Braces** mark the beginning and end of the function body.

Program Structure, Comments, and Running

```
#include <stdio.h>
/* This program prints one message. */
int main(void) {
// This is a single-line comment.
printf("Welcome to C!\n");
return 0;
}
```

Comments explain code and are ignored by the compiler. Use `//` for a single-line comment and `/* ... */` for a block comment.

Typical Terminal Commands

```
gcc main.c -o main
./main
```

The first command creates an executable named `main`. The second command runs it on Linux or macOS. On Windows, the command may be `main.exe`.

4. C Tokens

A token is a small meaningful part of a C program. The compiler reads a program as tokens.

Token	Meaning	Example
Keyword	A reserved word with a special meaning.	int, if, return
Identifier	A name given by the programmer.	total, main
Constant	A fixed value that does not change.	25, 3.14
String	Characters inside double quotation marks.	"Hello"
Operator	A symbol that performs an operation.	+, ==
Special symbol	A symbol that organizes code.	;, {}, ()

5. Variables and Constants

A variable is a named storage location. Its value can change while the program runs. A constant is a value that should not change.

```
#include <stdio.h>
int main(void) {
    int age = 18;
    const float pi = 3.14159f;
    age = 19;
    printf("%d %.2f\n", age, pi);
    return 0;
}
```

- **Declaration:** `int age;` reserves a place for an integer.
- **Initialization:** `int age = 18;` gives the first value.
- **Assignment:** `age = 19;` changes the value.
- **const:** prevents accidental changes to a value.

6. Data Types and Type Conversion

Type	Stores	Example
char	One character	'A'
int	Whole numbers	42
float	Decimal numbers	2.5f
double	More precise decimal numbers	2.5
void	No value	Used for functions with no return value

Type modifiers change the range or size of a type. Common modifiers are short, long, signed, and unsigned.

Specifier	Used with
%d or %i	int
%u	unsigned int
%c	char
%f	float in printf
%f	double in printf
%s	String
%zu	Result of sizeof

Conversion can be automatic or explicit. An explicit cast tells C to convert a value: `average = (float) total / count;`. The cast prevents integer division when a decimal answer is wanted.

C Prog

7. Input and Output

printf displays formatted output. **scanf** reads formatted input. Use `&variable` with `scanf` for most ordinary variables because C needs their address.

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("You are %d years old.\n", age);
    return 0;
}
```

getchar reads one character. **putchar** writes one character. **fgets** safely reads a line of text, including spaces. **puts** prints a string and adds a new line.

```
#include <stdio.h>
int main(void) {
    char letter;
    char name[40];
    letter = getchar();
    getchar();
    fgets(name, sizeof name, stdin);
    putchar(letter);
    puts(name);
    return 0;
}
```

The second `getchar` consumes the Enter key left in the input buffer. `sizeof name` gives the size of the character array, helping `fgets` avoid reading too much text.

8. Operators

Group	Operators	Example
Arithmetic	+ - * / %	a % 2
Relational	< > <= >= == !=	age >= 18
Logical	&& !	a > 0 && b > 0
Assignment	= += -= *= /= %=	total += price
Increment/decrement	++ --	count++
Conditional	? :	max = a > b ? a : b
Bitwise	& ^ ~ << >>	flags mask
Size	sizeof	sizeof(int)

Operator Precedence Basics

When an expression has many operators, C follows an order. Parentheses have the highest control. A useful simplified order is: parentheses, unary operators, multiplication/division/remainder, addition/subtraction, relational operators, equality operators, logical AND, logical OR, conditional operator, and assignment.

```
int answer = 2 + 3 * 4;
int clear_answer = (2 + 3) * 4;
```

The first answer is 14 because multiplication happens first. The second answer is 20 because parentheses happen first. Use parentheses when they make the meaning clearer.

ramming B

9. Decision Making

Decision statements choose which code should run. In C, a condition is treated as false when it is zero and true when it is nonzero.

-

```
#include <stdio.h>
int main(void) {
    int marks;
    scanf("%d", &marks);
    if (marks >= 90) {
        printf("Grade A\n");
    } else if (marks >= 50) {
        printf("Pass\n");
    } else {
        printf("Fail\n");
    }
    return 0;
}
```

- **if:** runs a block when its condition is true.
- **if-else:** chooses between two blocks.
- **else-if:** checks several conditions in order.
- **Nested if:** places one decision inside another decision.

switch

Use switch when one expression is compared with several fixed values.

```
int choice = 2;
switch (choice) {
case 1:
printf("Add\n");
break;
case 2:
printf("View\n");
break;
default:
printf("Unknown choice\n");
}
```

break stops the switch. Without it, execution may continue into the next case. **default** runs when no case matches.

10. Loops

Loops repeat a block of code. Use `while` when the condition is checked before each repetition, `do-while` when the block must run at least once, and `for` when the start, condition, and update fit together.

```
int i = 1;
while (i <= 3) {
printf("%d\n", i);
i++;
}

int j = 1;
do {
printf("%d\n", j);
j++;
} while (j <= 3);

for (int k = 1; k <= 3; k++) {
printf("%d\n", k);
}
```

break exits a loop. **continue** skips the rest of the current repetition and moves to the next repetition.

Nested Loops and a Pattern

```
for (int row = 1; row <= 3; row++) {  
    for (int col = 1; col <= row; col++) {  
        printf("* ");  
    }  
    printf("\n");  
}
```

This prints a triangle. The outer loop controls rows. The inner loop prints the stars in each row.

y Bhavya Ponduri

11. Functions

A function is a named block that performs one task. Functions make programs easier to read, test, and reuse.

```
#include <stdio.h>
int add(int a, int b);

int main(void) {
    int result = add(3, 4);
    printf("%d\n", result);
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

- **Declaration or prototype:** tells C the function name, return type, and parameters before use.
- **Definition:** contains the actual statements of the function.
- **Parameters:** are input values received by the function.
- **Return value:** is the result sent back with `return`.
- **Call:** runs the function, such as `add(3, 4)`.

C uses call by value for ordinary parameters. The function receives a copy, so changing the parameter does not change the original variable.

Recursion and Scope

Recursion happens when a function calls itself. A recursive function needs a base case so that it can stop.

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

The base case is $n \leq 1$. A local variable exists inside its block. A global variable is declared outside functions and can be used in many functions, but local variables are usually safer and easier to understand.

12. Arrays

An array stores several values of the same type in nearby memory locations. Indexing starts at zero.

```
#include <stdio.h>
int main(void) {
    int marks[3] = {70, 85, 90};
    for (int i = 0; i < 3; i++) {
        printf("%d\n", marks[i]);
    }
    return 0;
}
```

The valid indexes for `marks[3]` are 0, 1, and 2. Accessing index 3 is outside the array and causes undefined behavior.

Two-Dimensional Arrays and Searching

```
int table[2][2] = {{1, 2}, {3, 4}};
for (int r = 0; r < 2; r++) {
    for (int c = 0; c < 2; c++) {
        printf("%d ", table[r][c]);
    }
    printf("\n");
}
```

Linear search checks items one by one. **Sorting** arranges items. A simple bubble-sort step compares neighbors and swaps them when they are in the wrong order.

```
for (int pass = 0; pass < n - 1; pass++) {
    for (int i = 0; i < n - pass - 1; i++) {
        if (a[i] > a[i + 1]) {
            int temp = a[i];
            a[i] = a[i + 1];
            a[i + 1] = temp;
        }
    }
}
```

Common array mistakes are using the wrong index, forgetting the array size, reading uninitialized values, and writing past the last element.

BhavPath- Bhavya Ponduri

13. Strings

C has no separate built-in string type. A string is a character array ending with the special null character `'\0'`.

```
#include <stdio.h>
int main(void) {
    char name[30];
    printf("Enter your name: ");
    fgets(name, sizeof name, stdin);
    printf("Hello, %s", name);
    return 0;
}
```

The array must have room for every visible character plus `'\0'`. `fgets` is safer than an unbounded input function because it receives the array size.

Common String Functions

Include `string.h` to use these functions.

Function	Purpose
<code>strlen(s)</code>	Finds the length before <code>'\0'</code> .
<code>strcpy(d, s)</code>	Copies a string; ensure the destination is large enough.
<code>strcat(d, s)</code>	Adds one string to the end of another.
<code>strcmp(a, b)</code>	Compares two strings; zero means equal.

14. Pointers

A pointer is a variable that stores a memory address. The address operator `&` gets an address. The dereference operator `*` accesses the value at an address.

```
#include <stdio.h>
int main(void) {
    int number = 10;
    int *ptr = &number;
    printf("%d\n", *ptr);
    *ptr = 20;
    printf("%d\n", number);
    return 0;
}
```

- **int *ptr:** declares a pointer that should point to an integer.
- **&number:** gives the address of `number`.
- ***ptr:** reads or changes the integer stored at that address.

Pointer types should match the data they point to, such as `char *` for characters and `double *` for doubles. A null pointer points to no valid object and can be written as `NULL` after including an appropriate header.

Pointers with Arrays and Functions

An array name often acts like a pointer to its first element. A pointer parameter lets a function change an original variable.

```
void double_value(int *value) {
    *value = *value * 2;
}

int main(void) {
    int n = 5;
    double_value(&n);
    printf("%d\n", n);
    return 0;
}
```

The function receives the address of `n`, so changing `*value` changes the original value. Always initialize pointers and never dereference a null or invalid pointer.

15. Structures, Unions, Enumerations, and typedef

A **structure** groups different types under one name. Each member has its own storage.

```
#include <stdio.h>

struct Student {
    int id;
    char grade;
};

int main(void) {
    struct Student s = {101, 'A'};
    printf("%d %c\n", s.id, s.grade);
    return 0;
}
```

Use the dot operator, such as `s.id`, to access a member. Use `->` when accessing a member through a pointer to a structure.

A **union** stores different members in the same memory location. Only one member should be treated as valid at a time. An **enum** gives names to integer choices.

```
enum Day {MON, TUE, WED};
typedef unsigned int Number;
Number count = 10;
```

`typedef` creates a shorter name for an existing type. It does not create a new kind of memory.

16. Dynamic Memory Allocation

Dynamic memory is requested while the program is running. Include `stdlib.h` for these functions.

```

#include <stdio.h>
#include <stdlib.h>
int main(void) {
int *numbers = malloc(3 * sizeof *numbers);
if (numbers == NULL) {
return 1;
}
numbers[0] = 10;
numbers = realloc(numbers, 5 * sizeof *numbers);
free(numbers);
return 0;
}

```

Function	Purpose
<code>malloc</code>	Allocates uninitialized memory.
<code>calloc</code>	Allocates memory and sets its bytes to zero.
<code>realloc</code>	Changes the size of an existing allocation.
<code>free</code>	Returns allocated memory to the system.

Check allocation results, keep the pointer, do not use memory after `free`, and avoid losing the original pointer when using `realloc`.

17. File Handling

File handling lets a program save or read data. `fopen` opens a file and returns a file pointer. Always check for `NULL` and close the file with `fclose`.

```

#include <stdio.h>
int main(void) {
FILE *file = fopen("notes.txt", "w");
if (file == NULL) return 1;
fprintf(file, "C is useful.\n");
fclose(file);
return 0;
}

```

Mode	Meaning
"r"	Read an existing file.
"w"	Write; creates or replaces a file.
"a"	Append at the end of a file.
"r+"	Read and write an existing file.

Use `fscanf` or `fgets` for reading. Use `fprintf` or `fputs` for writing.

BhavPath- Bhavya Ponduri

18. Preprocessor Directives and Header Files

The preprocessor runs before the compiler. Its instructions begin with `#` and do not usually end with a semicolon.

- **#include:** copies declarations from a header file, such as `stdio.h`.
- **#define:** creates a simple macro replacement.
- **#if, #ifdef, #ifndef, #else, #endif:** control conditional compilation.

```
#include <stdio.h>
#define MAX_SCORE 100

int main(void) {
    printf("%d\n", MAX_SCORE);
    return 0;
}
```

Use parentheses in function-like macros to reduce surprises. For many cases, a `const` variable or function is safer than a macro.

19. Command-Line Arguments

Command-line arguments are values written after the program name. The standard form is `int main(int argc, char *argv[])`.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Arguments: %d\n", argc);
    if (argc > 1) {
        printf("First argument: %s\n", argv[1]);
    }
    return 0;
}
```

`argc` counts the program name and arguments. `argv` stores them as strings. Check `argc` before reading an element of `argv`.

20. Error Types and Debugging Tips

Error type	Meaning	Example
Syntax error	Code breaks a grammar rule.	Missing semicolon
Compile error	Compiler cannot translate the code.	Unknown variable
Linker error	Required function code is missing.	Missing library or definition
Runtime error	Problem occurs while running.	Divide by zero
Logic error	Program runs but gives a wrong result.	Wrong condition

- **Read from the first error:** later messages may be consequences.
- **Use small tests:** print values to check assumptions.
- **Trace the flow:** follow conditions and loop values by hand.
- **Use a debugger:** set a breakpoint and inspect variables step by step.
- **Compile with warnings:** warnings often reveal unsafe or unclear code.

21. Common Beginner Mistakes and Best Practices

Mistake	Better habit
Using = when comparison is needed	Use == for equality.
Forgetting & in scanf	Pass the address of ordinary variables.
Reading outside an array	Use indexes from 0 through size minus 1.
Using an uninitialized variable	Give variables a clear starting value.
Forgetting to free memory	Free every successful allocation once.
Writing very long functions	Split work into small, named functions.

Use meaningful names, indent consistently, keep comments useful, validate input, avoid duplicate code, and test both normal and unusual cases.

22. Small Projects

These projects combine the topics from the notes. Build a simple version first, then add improvements.

Project	Minimum features	Useful topics
Calculator	Read two numbers and an operator; show the answer.	Input, switch, operators, functions
Marksheet	Read marks, calculate total, average, and grade.	Arrays, loops, if-else, formatting
Number guessing	Choose a secret number and give higher/lower hints.	Loops, conditions, random numbers
Contact record	Store names and phone numbers in structures.	Structures, strings, arrays
Student record	Add, display, search, and save student records.	Structures, files, functions

Calculator Plan

1. Read the first number, operator, and second number.
2. Use `switch` to choose the operation.
3. Check for division by zero.
4. Print the result with a suitable format specifier.

Marksheet Plan

Store marks in an array, calculate the sum in a loop, find the average using a cast, and choose a grade with an `if-else` chain.

Number Guessing Plan

Generate a number, repeat input until the guess is correct, count attempts, and stop when the user succeeds or reaches a limit.

Contact and Student Records

Create a structure, use an array of structures, make menu functions, validate the record number, and use a file to save records between runs.

23. Important Viva and Interview Questions

Question	Short answer
What is C?	A compiled, general-purpose programming language.
What is a compiler?	A tool that translates source code into executable code.
What is <code>main</code> ?	The function where a C program starts.
What is a variable?	A named storage location whose value can change.
What is a pointer?	A variable that stores a memory address.
Why does an array start at zero?	The first element is at offset zero from the array start.
What is a string in C?	A character array ending with <code>'\0'</code> .
Difference between <code>while</code> and <code>do-while</code> ?	<code>do-while</code> runs its body at least once.
What does <code>break</code> do?	It exits the nearest loop or switch.
What does <code>continue</code> do?	It skips to the next loop repetition.
Why use <code>const</code> ?	It prevents a value from being changed through that name.
Why use <code>fclose</code> ?	It closes the file and releases related resources.

24. Practice Programs

Try each task first. Write the algorithm in plain English, then write the C code, test normal input, and test an edge case.

Level	Expected task
Starter	Print your name, age, and city on separate lines.
Starter	Read two integers and print their sum, difference, product, and remainder.
Starter	Check whether a number is even or odd.
Starter	Find the largest of three numbers using decisions.
Starter	Print the multiplication table of a number.
Loops	Find the sum of the first N natural numbers.
Loops	Reverse a number and check whether it is a palindrome.
Loops	Print a triangle, rectangle, and number staircase pattern.
Functions	Write functions for maximum, minimum, and average.
Arrays	Read an array, find its largest value, and calculate its average.
Arrays	Search for a value and print its index or a not-found message.
Arrays	Sort ten numbers in ascending order.
Strings	Count vowels, consonants, digits, and spaces in a line.
Pointers	Swap two integers using a function and pointers.
Structures	Store five students and print the student with the highest mark.
Files	Write names to a file, read them back, and

25. Final C Syntax Cheat Sheet

Need	Syntax
Include a header	<code>#include <stdio.h></code>
Program start	<code>int main(void) { }</code>
Variable	<code>int count = 0;</code>
Constant	<code>const double rate = 1.5;</code>
Output	<code>printf("%d\n", count);</code>
Input	<code>scanf("%d", &count);</code>
Condition	<code>if (condition) { }</code>
Loop	<code>for (int i = 0; i < n; i++) { }</code>
Function	<code>int add(int a, int b) { return a + b; }</code>
Array	<code>int values[5];</code>
Pointer	<code>int *p = &value;</code>
Structure	<code>struct Item { int id; };</code>
Dynamic memory	<code>int *p = malloc(n * sizeof *p);</code>
File	<code>FILE *f = fopen("data.txt", "r");</code>

26. Glossary

Word	Easy meaning
Algorithm	A step-by-step plan for solving a problem.
Array	A group of values of the same type.
Block	Statements surrounded by braces.
Compiler	A program that translates C source code.
Constant	A value intended not to change.
Declaration	A statement that introduces a name and type.
Dereference	Accessing the value stored at a pointer address.
Function	A reusable block that performs a task.
Header	A file containing useful declarations.
Identifier	A programmer-created name.
Initialization	Giving a variable its first value.
Pointer	A variable that stores an address.
Scope	The part of a program where a name can be used.
String	A character array ending in a null character.
Token	A meaningful unit read by the compiler.
Type	The kind of data a variable stores.

Final reminder: Understand the problem, make a small plan, write simple code, compile often, read warnings, and practice regularly.

6. Variables, Constants, Naming Rules, and Scope

A variable is a named place for a value. A constant is a value that should not change after it is set.



```
int age = 18;
final double TAX_RATE = 0.18;
age = 19;
System.out.println(age);
```

- **Declaration:** `int age;` creates a variable without a value.
- **Initialization:** `int age = 18;` gives the first value.
- **final:** prevents reassignment through that variable name.
- **Naming:** use letters, digits, underscore, or dollar sign; do not begin with a digit; do not use a keyword.
- **Style:** use `camelCase` for variables and methods, and `PascalCase` for classes.

Scope means the part of a program where a name can be used. A local variable belongs to a method or block. An instance variable belongs to an object. A class variable marked `static` belongs to the class.

7. Data Types and Type Conversion

Java has primitive types for simple values and reference types for objects.

Primitive	Stores	Example
byte, short, int, long	Whole numbers	<code>int count = 10;</code>
float, double	Decimal numbers	<code>double price = 2.5;</code>
char	One character	<code>'A'</code>
boolean	true OR false	<code>boolean ready = true;</code>

Reference types include classes, arrays, interfaces, and strings. A reference variable stores a link to an object rather than the object value itself.

```
double average = (double) total / count;
Integer boxed = 25;
int number = boxed;
```

The cast changes an integer calculation into a decimal calculation. Wrapper classes such as `Integer`, `Double`, and `Boolean` represent primitive values as objects. **Autoboxing** changes a primitive into its wrapper automatically, and **unboxing** changes it back.

8. Input and Output

print stays on the same line. **println** adds a new line. **printf** prints formatted values.

```
System.out.print("Name: ");
System.out.println("Java");
System.out.printf("Score: %.2f%n", 92.5);
```

Reading with Scanner

-

```
import java.util.Scanner;
Scanner input = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = input.nextInt();
System.out.println("Age = " + age);
input.close();
```

- **import:** makes the `Scanner` class available by its short name.
- **nextInt:** reads an integer. Use `nextDouble` for decimals and `nextLine` for a complete line.
- **BufferedReader:** can read text efficiently, usually with `InputStreamReader`. The text must be converted when a number is needed.

9. Operators and Precedence

Group	Operators	Example
Arithmetic	+ - * / %	a % 2
Relational	< > <= >= == !=	age >= 18
Logical	&& !	ready && paid
Assignment	= += -= *= /=	total += price
Unary	++ -- + - !	count++
Ternary	? :	max = a > b ? a : b
Bitwise and shift	& ^ ~ << >> >>>	flags mask
Type test	instanceof	item instanceof String

Multiplication, division, and remainder happen before addition and subtraction. Relational and logical operators come later. Assignment usually happens near the end. Use parentheses when they make the intention clearer.

10. Decision Making

Use decisions when a program must choose between actions.

-

```

if (marks >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Try again");
}

```

- **if:** runs code when its condition is true.
- **else-if:** checks another condition when earlier conditions are false.
- **nested if:** places one decision inside another.
- **switch:** compares one value with several cases.

```
switch (day) {  
    case 1 -> System.out.println("Monday");  
    case 6, 7 -> System.out.println("Weekend");  
    default -> System.out.println("Other day");  
}
```

The arrow form is a modern switch style. It does not need `break` after each case. Traditional switch statements use `case`, a colon, and usually `break`.

BhavPath- Bhavya Ponduri

11. Loops

Loops repeat code. Use `while` when the condition is checked first, `do-while` when the body must run once, and `for` when the start, test, and update fit together.

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
int[] numbers = {2, 4, 6};
for (int number : numbers) {
    System.out.println(number);
}
```

The enhanced `for` loop reads each array item without using an index. Use `break` to leave a loop and `continue` to skip to the next repetition.

Nested Loops and Patterns

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

The outer loop controls rows. The inner loop controls the number of stars in each row.

12. Methods

A method is a named block that performs a task. It can receive parameters and return a result.

```
static int add(int a, int b) {
    return a + b;
}
int result = add(3, 4);
```

- **Declaration:** the return type, name, and parameter list describe the method.
- **Parameters:** are input values received by the method.
- **return:** sends a value back to the caller. A `void` method returns no value.
- **static:** allows a method to be called through the class without first creating an object.
- **Overloading:** uses the same method name with different parameter lists.

```

static int total(int a, int b) { return a + b; }
static int total(int a, int b, int c) { return a + b + c; }
static int sum(int... values) {
    int result = 0;
    for (int value : values) result += value;
    return result;
}

```

Varargs `int...` allows zero or more integer arguments. Recursion is a method calling itself; it must have a base case that stops the calls.

13. Arrays

An array stores several values of one type. Its length is fixed after creation, and its first index is zero.

```

int[] marks = {70, 85, 90};
System.out.println(marks[0]);
for (int mark : marks) System.out.println(mark);

```

Two-Dimensional Arrays and Utilities

```

int[][] table = {{1, 2}, {3, 4}};
for (int[] row : table) {
    for (int value : row) System.out.print(value + " ");
    System.out.println();
}

```

Use `Arrays.sort(values)` to sort, `Arrays.toString(values)` to display, and `Arrays.binarySearch(values, key)` to search a sorted array.

- **Common mistakes:** using an invalid index, forgetting that length is fixed, reading an uninitialised item, and confusing `length` with a method call.

14. Strings

`String` stores text. Strings are objects and are immutable, which means their contents do not change after creation.

```
String name = "Java";
System.out.println(name.length());
System.out.println(name.toUpperCase());
System.out.println(name.substring(1, 3));
```

Useful methods include `length`, `charAt`, `contains`, `replace`, `trim`, and `split`. Compare text with `equals`, not `==`.

```
StringBuilder message = new StringBuilder("Hello");
message.append(" Java");
System.out.println(message);
```

Use `StringBuilder` when many text changes are needed. It changes one builder instead of creating many temporary `String` objects.

15. Object-Oriented Programming

A class is a plan for objects. An object is a real value created from a class. A constructor prepares a new object. The keyword `this` means the current object.

```
class Student {
    private String name;
    Student(String name) { this.name = name; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

- **private:** keeps a field available only inside its class.
- **public:** allows access from other classes.
- **Encapsulation:** protects fields and controls access through methods such as getters and setters.

16. Inheritance and Polymorphism

Inheritance lets one class receive useful members from another class. The child class uses `extends`. The keyword `super` refers to the parent class.

```
class Animal {  
    void speak() { System.out.println("Sound"); }  
}  
class Dog extends Animal {  
    @Override void speak() { System.out.println("Bark"); }  
}  
Animal pet = new Dog();  
pet.speak();
```

The last two lines show polymorphism: a parent reference can point to a child object, and the child version of an overridden method runs. An abstract class gives common design without always being created directly. An interface describes abilities that classes agree to provide.

- **final class:** cannot be extended.
- **final method:** cannot be overridden.
- **final variable:** cannot be assigned again.
- **Composition:** builds a class by giving it objects of other classes. Composition is often easier to change than deep inheritance.

17. Packages, Imports, and Access Control

A package groups related classes and helps avoid name conflicts. An import lets a class use a type by its short name.

```
package school.records;  
import java.util.ArrayList;  
public class StudentRecord { }
```

Modifier	Access
private	Only inside the same class.
default	Inside the same package.
protected	Same package and suitable subclasses.
public	From any package that can use the class.

Keep package names simple and place source files in folders that match their package names.

18. Exception Handling

An exception is a problem represented as an object. It can interrupt normal execution. Handle expected problems clearly instead of hiding them.

```
try {
    int result = 10 / number;
} catch (ArithmeticException ex) {
    System.out.println("Cannot divide by zero.");
} finally {
    System.out.println("Finished.");
}
```

- **try:** contains code that may fail.
- **catch:** responds to a matching exception type.
- **finally:** usually runs whether an exception happened or not.
- **throw:** creates and sends an exception now.
- **throws:** declares that a method may pass an exception to its caller.

```
class AgeException extends Exception {
    AgeException(String message) { super(message); }
}
```

Catch specific exceptions, give useful messages, do not use empty catch blocks, and do not use exceptions as normal program control.

19. Collections Framework

Collections store groups of objects and can grow or shrink. Choose a collection based on the job.

Collection	Useful for	Simple idea
ArrayList	Fast access by index.	Resizable list.
LinkedList	Frequent additions or removals.	Linked nodes.
HashSet	Unique values.	Fast lookup, no sorted order.
TreeSet	Unique sorted values.	Maintains order.
HashMap	Key and value pairs.	Find a value by key.

```
List<String> names = new ArrayList<>();
names.add("Asha");
for (String name : names) System.out.println(name);
Map<String, Integer> marks = new HashMap<>();
marks.put("Asha", 90);
System.out.println(marks.get("Asha"));
```

Use an enhanced for loop, an `Iterator`, or collection methods to iterate. Avoid changing a collection inside a loop unless the method safely supports it.

20. Generics

Generics tell Java what type a collection or class should store. They give type safety and reduce manual casting.

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
Box<String> box = new Box<>();
box.set("Notes");
```

The letter `T` is a type placeholder. A generic method can use its own type parameter, for example `static <T> void print(T value)`. Primitive types use wrapper classes in generics, such as `List<Integer>`.

21. File Handling and I/O

The modern file API uses `Path` and `Files`. The older `File` class is still found in many programs.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
Path path = Path.of("notes.txt");
Files.writeString(path, "First line\n".formatted());
Files.writeString(path, "Second line\n".formatted(),
    java.nio.file.StandardOpenOption.APPEND);
```

Use `Files.readString` to read a small text file and `Files.readAllLines` to read lines. For larger files, use buffering or a stream of lines. **Try-with-resources** closes resources automatically.

```
try (var reader = Files.newBufferedReader(path)) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException ex) {
    System.out.println("Could not read the file.");
}
```

22. Date and Time with `java.time`

The `java.time` package gives clear classes for dates and times.

```
LocalDate today = LocalDate.now();
LocalDate exam = LocalDate.of(2026, 6, 15);
long days = ChronoUnit.DAYS.between(today, exam);
System.out.println(today);
```

- **LocalDate**: date without a time.
- **LocalTime**: time without a date.
- **LocalDateTime**: date and time without a time zone.
- **ZonedDateTime**: date and time with a time zone.
- **DateTimeFormatter**: formats a date or time for display.

Date and time objects are immutable. Methods such as `plusDays` return a new value instead of changing the old one.

23. Enums, Records, Nested Classes, and Annotations

An enum represents a fixed group of named choices.

```
enum Level { LOW, MEDIUM, HIGH }  
Level current = Level.MEDIUM;  
record Point(int x, int y) { }  
Point point = new Point(2, 3);
```

An enum improves readability and prevents unrelated values. A record is a short way to create a class mainly used to hold data; Java supplies accessors, equality, and useful methods. A nested class is declared inside another class and can help keep related code together.

An annotation gives information to the compiler or tools. For example, `@Override` shows that a method should replace a parent method. An annotation does not automatically change program logic.

24. Lambda Expressions and Functional Interfaces

A lambda is a short way to write behaviour that can be passed around. A functional interface has one abstract method.

```
java.util.function.Predicate<Integer> even = n -> n % 2 == 0;  
System.out.println(even.test(8));
```

The parameter is `n`, the arrow separates the parameter from the expression, and the expression returns a boolean. Common functional interfaces include `Predicate`, `Function`, `Consumer`, and `Supplier`.

25. Streams

A stream processes values in a readable sequence. A stream does not usually store data; it describes operations on a source such as a list.

```
List<String> names = List.of("Mina", "Arun", "Meera");  
List<String> result = names.stream()  
    .filter(name -> name.length() > 4)  
    .map(String::toUpperCase)  
    .sorted()  
    .toList();
```

- **filter:** keeps values that pass a test.

- **map:** changes each value.
- **sorted:** orders values.
- **collect or toList:** creates a result collection.

Keep stream operations simple while learning. A normal loop can be clearer when the logic has many steps.

26. Multithreading Basics

A thread is a path of work that can run along with other work. Start with small examples and prefer safe library tools when building real applications.

```
Runnable task = () -> {
    System.out.println("Working");
};
Thread worker = new Thread(task);
worker.start();
```

Use `start` to begin a new thread. Calling `run` directly simply runs the method on the current thread. `Thread.sleep` pauses a thread and may require exception handling.

When several threads use the same data, a race condition can occur. Synchronization controls access so that only one suitable action changes shared data at a time. Prefer immutable values, clear ownership, and higher-level concurrency utilities instead of making every method synchronized.

27. JDBC and Database Basics

JDBC is the standard Java approach for connecting to a relational database. A program normally uses a driver, a connection, a prepared statement, and a result set.

```
String sql = "SELECT name FROM students WHERE id = ?";
try (var connection = DriverManager.getConnection(url, user, password);
    var statement = connection.prepareStatement(sql)) {
    statement.setInt(1, 10);
    try (var rows = statement.executeQuery()) {
        while (rows.next()) System.out.println(rows.getString("name"));
    }
}
```

The question mark is a parameter placeholder. A prepared statement is safer than joining user text into SQL. This example is a high-level pattern; a real program needs a database driver, connection details, and secure credentials.

28. Command-Line Arguments and Environment Variables

Arguments are values written after the program name. They arrive in the `args` array.

```
public static void main(String[] args) {
    if (args.length > 0) {
        System.out.println("Hello, " + args[0]);
    }
}
```

Read an environment variable with `System.getenv("NAME")`. Do not store passwords or private keys directly in source code. Check for missing arguments and missing environment values before using them.

29. Error Types and Stack Traces

Error	Meaning	Example
Syntax or compile error	Source code breaks a Java rule.	Missing semicolon
Runtime exception	Problem occurs while running.	Invalid index
Logic error	Program runs but gives a wrong answer.	Wrong condition
Input error	Input is not in the expected form.	Text instead of an integer

A stack trace usually shows the exception name, message, and method calls. Read the first useful line from your own source code, not only the last line. Check the file name, line number, input values, and most recent change.

- **Reproduce:** find a small input that shows the error.
- **Inspect:** print or watch important values.
- **Reduce:** remove unrelated code until the problem is clear.
- **Fix the cause:** do not simply hide the exception.

30. Common Beginner Mistakes and Best Practices

Mistake	Better habit
Comparing Strings with ==	Use equals for content.
Using an invalid array index	Check indexes from zero to length minus one.
Ignoring input errors	Validate input and show a useful message.
Making one very long method	Split work into small named methods.
Keeping fields public	Use private fields and controlled methods.
Swallowing exceptions	Catch only what you can handle.

Use meaningful names, keep indentation consistent, compile often, write small tests, avoid duplicate code, close resources, and explain unusual decisions with short comments.

31. Small Projects

Build a small version first. Then add validation, better menus, and file saving.

Project	Minimum features	Useful topics
Calculator	Read two numbers and an operation.	Scanner, switch, methods
Marksheet	Read marks, total, average, and grade.	Arrays, loops, conditions
Number guessing	Give higher or lower hints.	Loops, Random, input
Contact manager	Add, search, edit, and display contacts.	Classes, lists, strings
Student record	Store students and save them to a file.	Objects, files, exceptions
To-do list	Add tasks, mark them done, and list tasks.	ArrayList, classes, dates

Project Planning Checklist

- ☐ Write the problem in plain English.
- ☐ List the data the program must store.
- ☐ Design small methods for each action.
- ☐ Validate normal and unusual input.
- ☐ Test one feature before adding the next feature.
- ☐ Save data only after the basic version works.

32. Important Viva and Interview Questions

Question	Short answer
What is Java?	A general-purpose, object-oriented programming language.
What is the JVM?	The machine that runs Java bytecode.
What is the JDK?	Tools for developing Java programs.
What is a class?	A plan used to create objects.
What is an object?	A value created from a class.
What is a constructor?	A special member used when an object is created.
What is inheritance?	A child class receiving features from a parent class.
What is overriding?	Replacing an inherited method with a child version.
What is overloading?	Using one method name with different parameters.
Why use encapsulation?	To protect data and control how it is changed.
What is an exception?	An object that represents an unusual problem.
Why use generics?	To make types clear and reduce unsafe casts.
What is a lambda?	A short expression that represents an action.
What is a stream?	A sequence of operations for processing data.

33. Practice Programs

Try each task before looking at another solution. Write the algorithm first, then code it and test an edge case.

Level	Task
Starter	Print your name, age, and city on separate lines.
Starter	Read two numbers and print sum, difference, product, and quotient.
Starter	Check whether a number is even or odd.
Starter	Find the largest of three numbers.
Decisions	Read marks and print a grade using if-else.
Loops	Print a multiplication table.
Loops	Reverse a number and test whether it is a palindrome.
Loops	Print a triangle and a number staircase.
Methods	Write methods for maximum, minimum, and average.
Arrays	Find the largest value and average of an array.
Arrays	Search for a value and print its index.
Strings	Count vowels, consonants, digits, and spaces.
Objects	Create a Student class with a constructor and getters.
Collections	Store names in a list and remove duplicate names.
Files	Write names to a file, read them back, and append one name.
Streams	Filter even numbers, square them, sort them, and collect the result.

34. Final Java Syntax Cheat Sheet

Need	Syntax
Class	<code>class Student { }</code>
Main method	<code>public static void main(String[] args) { }</code>
Variable	<code>int count = 0;</code>
Constant	<code>final double RATE = 1.5;</code>
Output	<code>System.out.println(value);</code>
Input	<code>int n = scanner.nextInt();</code>
Condition	<code>if (condition) { }</code>
Switch	<code>case 1 -> action();</code>
Loop	<code>for (int i = 0; i < n; i++) { }</code>
Method	<code>int add(int a, int b) { return a + b; }</code>
Array	<code>int[] values = {1, 2, 3};</code>
List	<code>List<String> names = new ArrayList<>();</code>
Exception	<code>try { } catch (Exception ex) { }</code>
File	<code>Path path = Path.of("data.txt");</code>

35. Glossary

Word	Easy meaning
Array	A fixed group of values of one type.
Bytecode	The intermediate code created for the JVM.
Class	A plan for creating objects.
Constructor	Code that prepares a new object.
Encapsulation	Protecting data behind controlled methods.
Exception	An object that reports an unusual problem.
Field	A variable belonging to a class or object.
Generics	Type information written with a class or method.
Inheritance	A child class receiving features from a parent.
Interface	A contract describing methods a class can provide.
JDK	Tools used to develop Java programs.
JVM	The virtual machine that runs bytecode.
Lambda	A short expression representing an action.
Method	A named block that performs a task.
Object	A value created from a class.
Package	A group of related classes.
Polymorphism	One reference working with different object forms.
Reference	A variable that points to an object.
Scope	The part of code where a name is available.
Stream	A sequence of operations for processing

	data.
Variable	A named place whose value can change.

Final reminder: Understand the problem, make a small plan, write simple code, compile often, read the error message, and practise regularly.

A Simple Study Path

Start with variables, input, conditions, loops, methods, arrays, and strings. Then practise classes, objects, inheritance, exceptions, collections, and files. After that, explore lambdas, streams, threads, and databases one small example at a time.

BhavPath- Bhavya Ponduri